



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

Lenguaje y compiladores
Sección 01

Msc. Félix Márquez
fmarquez@e.uneg.edu.ve
Periodo lectivo: 2026-I

Tema 4 (Análisis Léxico)

- 4.1 Autómatas finitos y expresiones regulares.
- 4.2 Construcción de autómatas finitos a partir de gramáticas regulares.
- 4.3 Autómatas de pila
- 4.5 Construcción de analizadores léxicos desde el autómata,
- 4.6 Construcción del analizadores léxicos mediante metacompiladores.

Evaluación:
informes 15 defensa 10 (25 ptos)

Síntesis.

En este tema se materializa en gran medida los conceptos del tema anterior tanto en la continuación del estudio de las **máquinas teóricas (autómatas)** como con su vinculación con las gramáticas de libre contexto (GLC). En particular la creación del software: análisis léxico (*lexer*), 1) escribiendo el *lexer* desde “cero”, desde la base de la teoría de estas máquinas para un lenguaje seleccionado L y 2) utilizando una herramienta que escribiendo la descripción adecuada que representan el lenguaje L que deseamos y esta nos codifique el programa *lexer*:



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

En ambos casos el *lexer* tiene como entrada un programa P escrito en lenguaje L y el *lexer* indicará si P tiene errores léxicos.

Ahora una mirada mas al interior del *lexer*; y sobre la base que hemos revisado durante las exposiciones (defensas de informes de investigación), todo compilador requiere de un mecanismo inicial que evalúe el programa fuente y que en primera instancia determine la valides de las estructuras léxicas asociadas a un lenguaje formal L. En términos operativos, el programa fuente o entrada, es una lista de caracteres *string*, en la cual está contenido dicho programa, es trabajo de este mecanismo reconocer estos caracteres en la entrada: nueva linea, fin de archivo, espacios, tabuladores, letras, números, símbolos, entre otros. Además, identifica patrones en los caracteres leído, de modo que, puede identificar componentes léxicos del lenguaje: identificadores, operadores, números enteros, números reales, palabras reservadas, etc. El proceso de identificar estos componentes léxicos se llama tokenización y al valor que pueda tomar el token en un momento determinado lexema, ejemplo:

token	lexema
tk_identificador	Valor1 acum contador es_primo
tk_operador_matematico	+ - * /
tk_identificador_relacional	= >= <= <>

Es interesante como se reduce el trabajo con la **tokenización** ya que es posible: 1) aplicar reglas o **patrones** para reconocer los lexemas de cada token, 2) aplicar reglas de orden de aparición de cada token en una cadena de caracteres (programa), este proceso de aparición de los token se enmarcan en la noción de reglas sintácticas y se validarán en el próximo proceso del compilador (analizador sintáctico) el cual responde a una estructura gramatical tipo GLC (gramática de libre contexto), vista en el tema 3.

Los **patrones** o formas de estos componentes léxicos están asociados de forma directa a un L definido por el usuario y el mecanismo antes mencionado es diseñado en base a L. En tal sentido, en el analizador léxico, los programas, funciones o procedimientos están escritos para reconocer o rechazar la entrada (programa fuente) que no esté acorde con el lenguaje en cuestión. En otras palabras, el mecanismo del cual hablamos es un analizador léxico o programa capaz de leer una entrada en forma de programa fuente (escrito en términos de L) e informar si hay errores léxicos. En detalle, este analizador es el encargado de la lectura carácter a carácter y progresivamente reconocer si la entrada se ajusta a estructuras lexicas de L. De modo que, un



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

analizador léxico contiene: 1) un lector de la entrada carácter a carácter y 2) una función que verifica si un carácter o conjunto de ellos se ajusta a un patrón que define un componente léxico (token). En este último punto, requiere ser creado en términos de un programa de autómata finito determinístico del lenguaje L. Es suficiente un autómata finito para reconocer componentes léxicos de un lenguaje formal, toda vez que forman parte según jerarquía de Chomsky a los lenguajes regulares. El autómata que reconoce el lenguaje debe ser creado y luego escribir el programa que lo lleve a cabo, no es una tarea tan complicada, una vez teniendo el autómata, se implementa su tabla de transiciones de estado dentro de un bucle de lectura, esta a su vez está condicionada a la lectura carácter a carácter hasta el fin del archivo fuente.

Lo antes expuesto es una forma simple, desde “cero”, en la construcción de un analizador léxico. Es preciso aclarar que este autómata tendría diferentes estados de aceptación, uno por cada componente léxico o *tokens* y la cadena de entrada que originó dicho estado de aceptación *lexema*.

La ejecución del autómata no terminaría con un estado de aceptación, ya que este debe continuar leyendo hasta el final de la entrada. Es por ello que el autómata por cada estado final, es enviado de forma automática al estado inicial. Al final de la entrada, si se hace todo correcto en la construcción del autómata, se debe crear la tabla de transición y codificar el autómata en el lenguaje de programación de su preferencia.

Un aspecto importante con relación al esquema anterior en el desarrollo de un analizador léxico, es el control de errores, en primera instancia el analizador pudiera ser capaz de parar al primer error de lectura; si encontramos un compilador con un analizador léxico con estas características, entonces, este solo mostraría un error y luego pararía la ejecución. En este esquema, tal situación se corregiría si se construye el analizador con un autómata con estados fallidos y luego para la próxima entrada o carácter leído, se iniciaría desde el estado inicial del autómata. Claro está, son estos estados fallidos, donde se puede indicar, informar al usuario con cierto detalle el error léxico cometido.

Finalmente, para saltar la complejidad del desarrollo de AFD, una implementación interesante de los automatas, es su representación en expresiones regulares (regex) que nos facilita en términos de programación y basándose en patrones léxicos, la detección o reconocimientos de lexemas. Por otro lado, esta la utilización de herramientas que nos permitan construir analizadores léxicos ejecutables capaces de reconocer componentes léxicos que no se encuentren en un programa, sabemos que cuando esto ocurre decimos que el programa tiene errores léxicos, ejemplo: sin en lenguaje C escribimos **wile** en vez de **while**, o **fro** en vez de **for** detectará a **wile** y **fro** como dos errores léxicos porque son identificadores no conocidos para lenguaje C.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

Lectura para ejemplo en concreto:

"Diseño e Implementación de Analizadores Léxicos: Desde la Teoría de Autómatas hasta la práctica con Metacompiladores".

1.- Introducción

En este documento se profundiza en los conceptos teóricos y prácticos relacionados con el diseño e implementación de analizadores léxicos (lexers), tanto desde un enfoque manual basado en expresiones regulares (regex) como utilizando herramientas de metacompilación. El análisis léxico es una fase fundamental en el proceso de compilación, ya que se encarga de transformar una secuencia de caracteres en una secuencia de tokens, a este proceso se le denomina tokenización, aspecto fundamental en un lexer, los tokens, son las unidades básicas en el procesamiento de lenguaje (tanto lenguaje formal como natural) . Este proceso es esencial para garantizar que el programa fuente esté libre de errores léxicos antes de pasar a las siguientes fases del compilador, como el análisis sintáctico y semántico. Un **token** refiere a un concepto y que puede ser ocupado o tomar valor (**lexema**), ejemplo token: tk_identificador lexemas posibles: valor1, cont, suma, contador, etc; token: tk_while, lexema: while. Verá que los token no necesariamente deben iniciar con "tk", mas adelante se profundizará al respecto.

El documento está estructurado de la siguiente manera: primero, se revisan los fundamentos teóricos de los autómatas finitos y su relación con las gramáticas de libre contexto (GLC). Luego, se presenta ejemplo práctico de un analizador léxico implementado desde cero para un lenguaje utilizado en las interfaces de Linux. Finalmente, se muestra cómo se puede lograr el mismo resultado utilizando herramientas de metacompilación, como Flex, y se comparan ambos enfoques.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

2. Repaso Teóricos necesario para el *lexer*:

2.1. Autómatas Finitos y Lenguajes Regulares.

Los autómatas finitos son máquinas teóricas que reconocen lenguajes regulares. Un autómata finito determinístico (AFD) es una tupla $(Q, \Sigma, \delta, q_0, F)$, donde:

Q es un conjunto finito de estados.

Σ es un alfabeto finito de símbolos de entrada.

$\delta: Q \times \Sigma \rightarrow Q$ es la función de transición.

$q_0 \in Q$ es el estado inicial.

$F \subseteq Q$ es el conjunto de estados de aceptación.

Los lenguajes regulares son aquellos que pueden ser reconocidos por un AFD y están en la base de la jerarquía de Chomsky. Estos lenguajes son fundamentales para el análisis léxico, ya que los tokens de un lenguaje de programación suelen ser expresiones regulares.

2.3. Proceso de Tokenización.

La tokenización es el proceso de convertir una secuencia de caracteres en una secuencia de tokens. Cada token tiene un tipo (por ejemplo, identificador, número, operador) y un lexema (el valor concreto del token). Por ejemplo, en el siguiente código:

c

```
int x = 10;
```

Los tokens serían:

- tk_tipo_dato (lexema:int)
- tk_identificador (lexema: x)
- tk_operador_asignacion (lexema: =)
- tk_numero (lexema: 10)



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

- tk_fin_de_linea (lexema: ;)

3.- Implementación de un Analizador Léxico para el Archivo interfaces de Debian.

En esta sección, se implementará un analizador léxico para el archivo de configuración de interfaces de red de Debian 11 (/etc/network/interfaces). Este archivo contiene configuraciones de red, como direcciones IP, máscaras de red, gateways, y otros parámetros. Además, se manejarán los comentarios (indicados por #), que no deben ser procesados por el analizador léxico.

Se implementarán dos versiones del analizador léxico: Desde cero, utilizando Python y Usando Flex, una herramienta clásica de metacompilación.

3.1.- Implementación desde Cero en Python.

3.1.2.- Descripción del Lenguaje

El archivo interfaces tiene una estructura específica:

- Comentarios: Las líneas que comienzan con # son comentarios y deben ser ignoradas.
- Directivas: Palabras clave como auto, iface, address, netmask, gateway, etc.
- Valores: Direcciones IP, máscaras de red, nombres de interfaces, etc.

ejemplo del archivo:

3.1.2.2.- Implementación del Lexer en Python.

A continuación, se presenta un analizador léxico implementado desde cero en Python. Este lexer ignora los comentarios y reconoce las directivas y valores relevantes.

```
import re
```



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

```
1
2  # Definición de los tokens actualizada
3  tokens = [
4      ('AUTO', r'\bauto\b'),      # Solo al inicio de línea
5      ('IFACE', r'\biface\b'),    # Solo al inicio de línea
6      ('VLAN_RAW_DEVICE', r'\bvlan_raw_device\b'),
7      ('ADDRESS', r'\baddress\b'),
8      ('NETMASK', r'\bnetmask\b'),
9      ('GATEWAY', r'\bgateway\b'),
10     ('NETWORK', r'\bnetwork\b'),
11     ('BROADCAST', r'\bbroadcast\b'),
12     ('INET', r'\binet\b'),
13     ('LOOPBACK', r'\bloopback\b'),
14     ('STATIC', r'\bstatic\b'),
15     ('DHCP', r'\bdhcp\b'),
16     ('IP', r'\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}'),
17     ('INTERFACE_NAME', r'[a-zA-Z0-9_-]+'),
18     ('NEWLINE', r'\n'),
19     ('SKIP', r'[\t]+'),
20     ('COMMENT', r'#. *'),
21     ('MISMATCH', r''),
22 ]
23
24 def lexer(input_text):
25     token_regex = '|'.join(f'(?P<{name}>{pattern})' for name, pattern in tokens)
26     line_num = 1
27     line_start = 0
28     for mo in re.finditer(token_regex, input_text, re.MULTILINE):
29         kind = mo.lastgroup
30         value = mo.group(kind)
31         if kind == 'NEWLINE':
32             line_start = mo.end()
33             line_num += 1
34         elif kind == 'SKIP' or kind == 'COMMENT':
35             continue # Ignorar espacios, tabulaciones y comentarios
36
```



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

```
37     elif kind == 'MISMATCH':
38         raise RuntimeError(f'{value!r} unexpected on line {line_num}')
39     else:
40         column = mo.start() - line_start
41         yield kind, value, line_num, column
42
43 # Resto del código sin cambios...
44
45 def cargar_archivo(nombre_archivo):
46     try:
47         with open(nombre_archivo, 'r') as file:
48             return file.read()
49     except FileNotFoundError:
50         print(f"Error: El archivo '{nombre_archivo}' no fue encontrado. Sorry!")
51         return None
52     except Exception as e:
53         print(f"Error al leer el archivo: {e}")
54         return None
55
56 nombre_archivo = 'interfaces'
57
58 input_text = cargar_archivo(nombre_archivo)
59
60 if input_text is not None:
61     for token in lexer(input_text):
62         print(token)
```

1.3. Explicación del Código.

Como se indico antes la idea se centra en la tokenización y en ese particular la expresiones regulares juegan un papel preponderante.

- Expresiones Regulares: Cada token está definido por una expresión regular. Por ejemplo, AUTO se define como `\bauto\b`, lo que significa que debe coincidir con la palabra "auto" al inicio de una línea, es posible



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

entrenar las habilidades en la creación de expresiones regulares mediante editores online regex (<https://regexr.com/> o <https://regex101.com/>)

- Ignorar Comentarios: Las líneas que comienzan con # se ignoran completamente.

1.4. Ejecución.

Archivo a analizar:

```
/etc/network/interfaces
# This file describes the network interfaces
auto lo eth0
iface lo inet loopback

iface eth0 inet static
    address 192.168.1.1
    netmask 255.255.255.0
    gateway 192.168.1.25
```

Se tiene la siguiente salida, en el cual se localizan los tokens encontrados.

En la salida se identificas los tokens localizados.

```
('AUTO', 'auto', 2, 0)
('INTERFACE_NAME', 'lo', 2, 5)
('INTERFACE_NAME', 'eth0', 2, 8)
('IFACE', 'iface', 3, 0)
('INTERFACE_NAME', 'lo', 3, 6)
('INET', 'inet', 3, 9)
('LOOPBACK', 'loopback', 3, 14)
('IFACE', 'iface', 5, 0)
('INTERFACE_NAME', 'eth0', 5, 6)
('INET', 'inet', 5, 11)
('STATIC', 'static', 5, 16)
('ADDRESS', 'address', 6, 4)
('IP', '192.168.1.1', 6, 12)
('NETMASK', 'netmask', 7, 4)
```



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

```
('IP', '255.255.255.0', 7, 12)
('GATEWAY', 'gateway', 8, 4)
('IP', '192.168.1.254', 8, 12)
```

Un ejemplo adicional, considere que el archivo se escriba la palabra “ddress”, como verá esta palabra no es reconocido por la regla regex de la línea de código No.7. pero si identificado como lexema del token INTERFACE_NAME, ya que pasa la prueba de la regex 17.

Ejercicio práctica: por cada expresión regular cree su correspondiente AFD.

3.2. Implementación usando metacompilador (Flex)

3.2.1. Descripción del Archivo de Entrada para Flex

A continuación, se muestra cómo se puede implementar el mismo analizador léxico utilizando Flex. El archivo de especificación para Flex tiene la extensión .l (letra ele de lexer)

```
Lexer.l
1  %{
2  #include <stdio.h>
3  %}
4
5  %option noyywrap
6
7  auto      { printf("TOKEN: AUTO, Lexema: %s\n", yytext); }
8  iface     { printf("TOKEN: IFACE, Lexema: %s\n", yytext); }
9  address   { printf("TOKEN: ADDRESS, Lexema: %s\n", yytext); }
10 netmask   { printf("TOKEN: NETMASK, Lexema: %s\n", yytext); }
11 gateway   { printf("TOKEN: GATEWAY, Lexema: %s\n", yytext); }
12 network   { printf("TOKEN: NETWORK, Lexema: %s\n", yytext); }
13 broadcast { printf("TOKEN: BROADCAST, Lexema: %s\n", yytext); }
14 vlan_raw_device { printf("TOKEN: VLAN_RAW_DEVICE, Lexema: %s\n", yytext); }
15 inet      { printf("TOKEN: INET, Lexema: %s\n", yytext); }
```



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

```
16 loopback      { printf("TOKEN: LOOPBACK, Lexema: %s\n", yytext); }
17 static        { printf("TOKEN: STATIC, Lexema: %s\n", yytext); }
18 dhcp          { printf("TOKEN: DHCP, Lexema: %s\n", yytext); }
19 [0-9]+\.[0-9]+\.[0-9]+\.[0-9]+ { printf("TOKEN: IP, Lexema: %s\n", yytext); }
20 [a-zA-Z0-9_-]+ { printf("TOKEN: INTERFACE_NAME, Lexema: %s\n", yytext); }
21 \n            { /* Ignorar nuevas líneas */ }
22 [\t]+         { /* Ignorar espacios y tabulaciones */ }
23 #.*           { /* Ignorar comentarios */ }
24 .             { printf("ERROR LÉXICO: Carácter no reconocido: %s\n", yytext); }
25
26 %%
27
28 int main(int argc, char **argv) {
29     if (argc > 1) {
30         FILE *file = fopen(argv[1], "r");
31         if (!file) {
32             perror("Error al abrir el archivo");
33             return 1;
34         }
35         yyin = file;
36     }
37     yylex();
38     return 0;
39 }
```

3.2.2. Generación y Compilación del Lexer con Flex

Generar el Lexer:

```
bash
flex archivo_lexico.l
```

Compilar el Lexer:

```
bash
gcc lex.yy.c -o analizador_lexico
```



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

Ejecutar el Lexer:

bash

./analizador_lexico interfaces

3.2.3. Salida Esperada

Para el archivo de entrada proporcionada se tiene:

Salida

TOKEN: AUTO, Lexema: auto
TOKEN: INTERFACE_NAME, Lexema: lo
TOKEN: INTERFACE_NAME, Lexema: eth0
TOKEN: IFACE, Lexema : iface
TOKEN: INTERFACE_NAME, Lexema: lo
TOKEN: INET, Lexema : inet
TOKEN: LOOPBACK, Lexema: loopback
TOKEN: IFACE, Lexema: iface
TOKEN: INTERFACE_NAME, Lexema: eth0
TOKEN: INET, Lexema: inet
TOKEN: STATIC, Lexema: static
TOKEN: ADDRESS, Lexema : address
TOKEN: IP, Lexema: 192.168.1.1
TOKEN: NETMASK, Lexema: netmask
TOKEN: IP, Lexema: 255.255.255.0
TOKEN: GATEWAY, Lexema: gateway
TOKEN: IP, Lexema: 192.168.1.254

3.3. Comparación entre Ambos Enfoques.

regex: Proporciona un control total sobre el proceso de análisis léxico, pero requiere mas esfuerzos y es propenso a errores, en contraste el Metacompilador FLEX: facilita la creación rápida de analizadores léxicos, pero puede ser menos flexible en ciertos escenarios.

3.4. Conclusiones



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

Ambos enfoques son válidos para implementar un analizador léxico. La elección entre ellos dependerá de las necesidades específicas del proyecto y de la preferencia del desarrollador.

4.- Actividad del tema.

1) Defina y presente ejemplos de autónoma de pilas.

En el Informe indicar lo antes descrito, incorporar una conclusión de las utilidades del autómata de pila, aborde su importancia relacionado a su poder con relación a los AFD y AFND.

2) Construya un *lexer* para la verificación de archivos docker mediante expresiones regulares.

En el Informe debe indicar paso a paso la construcción de su lexer y 3 ejemplos de ejecución.

En el archivo comprimido entregar los códigos (fuentes y ejecutables).

3) Definir un lenguaje L subconjunto de lenguaje Rust, construya un lexer para L utilizando metacompilador.

- En el informe incorpore: 1) manual de usuario completo con sus propias palabras del metacompilador seleccionado, 2) descripción de su lenguaje L diseñado, 3) documentar proceso de creación de su lexer incorporar en repositorio con su respectivo archivo markdown de documentación y pasos a detalle de instalación o implementación del lexer.

4) Indague y reflexione en que lenguajes en el área de seguridad informática, pudiera aplicar un FLEX, presente el lenguaje y su respectiva tokenización.

Consideraciones adicionales .

El Líder enviar repositorio con el código e informe el cual debe contener información acerca de las actividades de la tarea, preste especial atención a que este debe documentar suficientemente los programas o soluciones de software construidos y tecnología utilizada, herramientas, detalle de instalación y ejecución



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
Lenguaje y compiladores

Evaluación entrega de informe y archivos,

Informe, código fuentes y compilables (para el caso que aplique) **5%**

Defensa **10%**. exposición grabada 10 min por participante, valide detalles como el sonido y nitidez de las láminas, para poder apreciar el dominio del contenido.

Entrega lunes 13/07/2026